

Train-to-Eval Distribution Shift in Deterministic Policy Execution

1 Problem Diagnosis from Evaluation Curves

A deterministic evaluation plateau at exactly 81 steps with return near -2 is consistent with an inactivity termination condition (“idle timeout” / “no progress”), where the vehicle fails to generate forward motion and is reset with penalty.

2 Why Stochastic Training Can Hide Deterministic Failure

Let the gas-action pre-tanh scalar be

$$u_{\text{gas}} = \mu_{\text{gas}} + \sigma_{\text{gas}} \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1). \quad (1)$$

The executed control is $a_{\text{gas}} = \tanh(u_{\text{gas}})$.

Training (stochastic sampling)

With early-policy parameters such as $\mu_{\text{gas}} = -0.2$, $\sigma_{\text{gas}} = 1.0$, and sampled noise $\epsilon = 1.5$:

$$u_{\text{gas}} = -0.2 + (1.0 \times 1.5) = 1.3, \quad a_{\text{gas}} = \tanh(1.3) \approx 0.86. \quad (2)$$

The car moves forward, so replay contains apparently successful transitions, even though this behavior is driven by stochastic variance rather than mean policy quality.

Evaluation (deterministic execution)

Deterministic testing removes sampling ($\epsilon = 0$), so action depends only on the mean:

$$u_{\text{gas}} = \mu_{\text{gas}} + 0 = -0.2, \quad a_{\text{gas}} = \tanh(-0.2) \approx -0.19. \quad (3)$$

Thus, the latent policy can command near-zero (or negative) throttle, producing no progress and triggering the 81-step inactivity termination.

3 Implementation Fixes

Fix 1: Pre-tanh Forward Bias (Action Asymmetry)

Add a positive bias to the gas channel before tanh during deterministic policy execution:

```
# Assuming action output is [Steer, Gas, Brake]
# action_mean is your 'mu'
```

```
# Add a +0.5 bias to the Gas pedal (Index 1)
forward_bias = torch.zeros_like(action_mean)
forward_bias[:, 1] = 0.5 # Bias the gas positively

# Now apply the Tanh squash
deterministic_action = torch.tanh(action_mean + forward_bias)
```

Equivalent form:

$$a_{\text{eval}} = \tanh(\mu_a + [0, 0.5, 0]). \quad (4)$$

This ensures baseline forward drive even when raw mean outputs are near zero.

Fix 2: Anti-idle Reward Shaping

Penalize low-speed inactivity directly in the environment wrapper:

```
# If the car is barely moving, unleash a massive penalty
if speed < 5.0 and step_count > 10:
    reward -= 5.0 # Make sitting still hurt mathematically!
```

This reshapes critic targets so idle behavior receives strongly negative value estimates.

Fix 3: Reduce Initial Exploration Scale

Lowering the log-standard-deviation ceiling (e.g., `LOG_STD_MAX`) or initial entropy temperature limits reliance on noise-driven motion, forcing the actor to increase μ_{gas} directly.

4 Reward Shaping Strategy (Root-Cause Resolution)

Sparse vs. Dense Reward Philosophy

Pure sparse reward formulations (e.g., reward only at race completion) are typically insufficient in continuous-control racing because early random policies almost never encounter terminal success. Practical training therefore uses dense reward signals that provide local guidance toward long-horizon objectives.

Reward-Hacking Risk

Reward shaping introduces optimization loopholes unless designed carefully. Typical racing failure modes include:

- **Spinning hack:** farming speed-like rewards without track progress,
- **Waypoint farming:** repeatedly crossing a local trigger in both directions,
- **Idle local optimum:** preferring no motion when crash penalties dominate.

Potential-Based Reward Shaping (PBRs)

To preserve policy consistency while densifying reward, use potential differences:

$$F(s_t, s_{t+1}) = \gamma\Phi(s_{t+1}) - \Phi(s_t). \quad (5)$$

If the agent remains stationary in state-space, potential change is near zero, reducing exploitability of static loops.

TrackMania-Oriented Reward Components

We define a three-term reward:

$$R_t = R_{\text{progress}} + R_{\text{idle}} + R_{\text{smooth}}, \quad (6)$$

$$R_{\text{progress}} = \Delta d_{\text{centerline}}, \quad (7)$$

$$R_{\text{idle}} = \begin{cases} -1.0, & \text{if speed} < 10 \text{ km/h,} \\ 0, & \text{otherwise,} \end{cases} \quad (8)$$

$$R_{\text{smooth}} = -\lambda_s (\text{steer}_t - \text{steer}_{t-1})^2. \quad (9)$$

Here, R_{progress} rewards forward displacement along the centerline (not raw speed), R_{idle} removes the AFK attractor, and R_{smooth} discourages high-frequency steering jitter.

Reference Implementation Sketch

```
def calculate_reward(self, current_state, previous_state, action, previous_action):
    # 1. Forward Progress (Potential-based)
    progress_reward = current_state.distance_on_centerline - previous_state.distance_on_centerline

    # 2. The Stick (Fixes the Deterministic Freeze!)
    if current_state.speed < 5.0: # Less than 5 km/h
        idle_penalty = -2.0
    else:
        idle_penalty = 0.0

    # 3. Smoothness Penalty (Prevents jitter)
    steering_diff = abs(action[0] - previous_action[0]) # Assuming index 0 is steering
    smoothness_penalty = -0.5 * (steering_diff ** 2)

    # Final Formula
    total_reward = progress_reward + idle_penalty + smoothness_penalty

    return total_reward
```

Impact on Deterministic Collapse

Before shaping, inactivity could dominate early deterministic returns when the policy estimated higher expected value for “not crashing” than for noisy exploration. Adding explicit idle penalty changes the critic landscape so that stationary behavior becomes a negative sink, forcing the actor to generate forward momentum and removing the 81-step deterministic freeze mode.

5 Thesis-Ready Statement

Because the continuous action policy relied disproportionately on stochastic variance σ to generate forward momentum during training, the deterministic mean μ remained near zero in the gas channel. During evaluation, removing stochastic sampling exposed this mismatch, yielding a zero-momentum regime and repeated 81-step inactivity termination. Introducing a pre-tanh positive gas bias aligned

deterministic control with the asymmetric physical requirements of racing and mitigated train-to-eval distribution shift.

Initially, the deterministic policy collapsed into an idle local optimum because the base reward function yielded higher expected return for inactivity than for early exploratory collisions. By introducing shaped rewards with an explicit idle penalty and progress-linked potential signal, the Q-value surface no longer favored stationary behavior, and deterministic evaluation recovered sustained forward motion.